

1 INTRODUCTION

Before we deep dive into the Pythonic content of this chapter, we must go back to the history of computer programming language or computer language. First, we need to know why a computer programming language is required. Although some of us already know the answer is very simple “**To automate a task**” or “**To reduce the workload of a human**”. For this, the first foundation of computer languages was started in the early 1940s and since then many programming languages have been designed with some specific purpose. The details of all computer languages other than Python are discussed in the first section of this chapter. The remaining part of the chapter belongs to Python and is sub-categorized as follows:

- The Section **Why Python?** highlights the intention of the Python developer along with language features.
- Next Section **Downside of Python** provides the real drawback of this language, as compared with other languages.
- Third Section **What Python can do?** highlights some of the important tasks, where Python is mostly used.
- The complete evaluation of Python language is highlighted in the next Section namely **Evaluation of Python**
- The most used Python libraries are listed in Section **Popular Python Libraries** so that any reader can target any specific library for his purpose of learning this language.
- Finally, the last Section **Python Implementations** includes the details of all the implementations of Python till today.

1.1 History of Computer Languages

Programming languages differ from natural languages in that natural languages are used to exchange information between humans, while programming languages are used by humans to interact with machines. Programming languages generally contain abstractions for defining and manipulating the data structures or controlling the flow of execution.

Most of the programming languages are text-based and mainly composed of two components syntax and semantics. Some of them are specified in a document called ISO and some are referenced such as Perl. The third category has a mix of two, with the basic language defined by a standard and extensions taken from the dominant implementation.

Early computers such as Colossus¹ were programmed without stored programs, simply by modifying

¹ https://en.wikipedia.org/wiki/Colossus_computer

their circuit or by setting banks of physical controls. After a few years, the first-generation (1GL) computer language was designed in the form of machine language, where each instruction is written in numeric form and executed the instructions in the hardware directly.

The second-generation languages (2GL) are also very close to machine architecture known as assembly language. These assembly languages are more human-readable form than 1GL and relieve the programmer of tedious and error-prone address calculations. In the 1950s between 1943 and 1945 the first high-level language Plankalkül was written by Konrad Zuse for the German Z3 computer.

In 1949 Khon Mauchly's Short Code² was the first high-level language developed for computers. Short Code statements are close to machine code represented in mathematical expression and much more in understandable form.

The first high-level compiled language Autocode was founded by Alick Glennie at the University of Manchester in 1952. A compiler is a system program used to translate a high-level instruction to machine language. The second auto code was designed by R. A. Brooker in 1954 known as Mark 1 Autocode. Both of these Autocodes were designed to work with Mark 1 computers only. The required auto code for the EDSAC³ 2 computer was founded by D. F. Hartley at the Mathematical Laboratory of the University of Cambridge. Similar to this a separate auto code for the Atlas 1 machine was founded at the University of Manchester.

The most popular general-purpose high-level programming language called FORTRAN⁴ was founded in 1954 at IBM by Jhon Backus. FORTRAN is still popular for high-performance computing and used for programs that benchmark and rank the world's fastest supercomputers.

Another early general purpose language FLOW-MATIC⁵ was founded by Grace Hopper and specifically designed to program in the first electronic computer UNIVAC I. Grace and her team members found it difficult to process business data related to customers using mathematical notations and in early 1955 they came up with specification for an English programming language and FLOW-MATIC become publicly available in 1959.

The increased use of high-level languages introduced another requirement in the programming paradigm. At this point in the early 1960s, there was a requirement for low-level system programming languages, which provided facilities to perform tasks that required direct access to hardware facilities with high-level control structures with error checking. Between the periods of 1960s to 1970s, most of the languages are now in use. Some of the present-era languages are inspired by them and some are limited in use. Table 1.1 provides a list of all such computer languages that exist at present along with their original purpose at the time of development and the programming paradigm they support.

Table 1.1 List of most popular programming languages with their purpose and programming paradigm

Language	Purpose	Paradigm	Year
Fortran	especially suited to numeric computation and scientific computing	Multi-paradigm: structured, imperative (procedural, object-oriented), generic, Array	1957
Lisp	Popular for artificial intelligence research	functional, procedural, reflective, meta	1960
APL	describing procedures in the processing		

² [https://en.wikipedia.org/wiki/Short_Code_\(computer_language\)](https://en.wikipedia.org/wiki/Short_Code_(computer_language))

³ <https://en.wikipedia.org/wiki/EDSAC>

⁴ <https://en.wikipedia.org/wiki/FORTRAN>

⁵ <https://en.wikipedia.org/wiki/FLOW-MATIC>

	of information		
PASCAL	Used to develop dynamic and recursive data structures	Imperative and structured	1970
Shell script	designed to be run by a Unix shell, a command-line interpreter.	Scripted event	1971
Prolog	Used for logic building in artificial intelligence along with theorem proving, expert systems, term rewriting, type systems, and automated planning.	Supports only logic building	1972
C	use in operating systems, device drivers, and protocol stacks, though decreasingly for application software.	Multi-paradigm as procedural and structured	1972
SQL	The domain-specific language used in RDBMS for data management and manipulating	Supports declarative paradigm	1974
C++	Mostly used for system programming and embedded systems, resource-constrained software, and large systems in mind, with performance, efficiency, and flexibility.	Supports procedural, imperative, functional, object-oriented, generic, modular	1979
Ada	Originally designed for embedded and real-time systems.	Supports structured, imperative, object-oriented, aspect-oriented, concurrent, array, distributed, generic, procedural, meta	1980
MATLAB	Mostly used for numeric computing. It allows the design of user interface and plotting of data as well.	functional, imperative, procedural, object-oriented, Array	1984
J	Is an array programming language mostly used for mathematical and statistical programming suitable for matrices operations.	Supports functional programming via its tactic programming feature. Also be used for both object and prototype-based programming.	1990
Python	Discussed later in this Chapter	object-oriented, procedural, functional, structured, reflective	1991
R	Mostly used for data mining, bioinformatics, statistics for data analysis	procedural, object-oriented, functional, reflective, imperative, Array	1993
HTML	For web development for webpage design	Markup language	1993
Java	It is a compiled language designed to write once and run anywhere. Mainly used for client-server web applications.	structured, imperative, object-oriented, aspect-oriented, concurrent, array, distributed, generic, procedural, meta	1995

PHP	Used for Web development	Supports imperative, object-oriented, procedural, reflective	Multi-paradigm: functional, object-oriented	1995
JavaScript	Mostly used for web development along with HTML and CSS	Supports event-driven, procedural, programming	functional, object-oriented	1995
Ruby	It was designed with an emphasis on programming productivity and simplicity.	functional, imperative, object-oriented, reflective		1995
C#	It was designed for developing software components suitable for deployment in distributed environments.	structured, imperative, object-oriented, event-driven, task-driven, functional, generic, reflective, concurrent		2000
Scala	It was designed as scalable and designed to grow with the demand of users.	concurrent, functional, imperative, object-oriented		2004
Go or Golang	Similar to C but with memory safety, garbage collection, structural typing, and CSP-style concurrency.	concurrent imperative, object-oriented		2009
Rust	Popular for system programming also offers features for functional programming	concurrent, functional, imperative, structured	generic, generic	2010
TypeScript	It is a superset of JavaScript and intended to be designed for the development of large applications and transpires into JavaScript	functional, generic, imperative, object-oriented		2012
Julia	Well-suited for numeric analysis and computational science	multiple dispatch (primary paradigm), procedural, functional, meta, multistage		2012

This list is taken from IEEE Spectrums⁶, which ranked top performing programming languages. At the time of writing this book some new languages are added or removed from this list. The key point that we need to note down from the evaluation of programming languages is, that every language needs to evolve with time and must add some unique programming features as compared to existing languages with rich library support.

Most important we need to note that at the time of writing this chapter Python is the most trading language followed by Java, C, JavaScript, and C++ as the top five. As of now, SQL is leading the Job market followed by Java, Python, JavaScript, and C#. I hope that as a writer of this book, I can answer this question why Python? Explained in the next Section.

Being an instructor for programming languages over the past 10 years my suggestion for all the readers is to go for a Python computer language with a goal and if you find the answer to why then it's time to start learning that language by writing small programs. Remember any computer language can't be fitted in just one book or syllabus, you have to go beyond that.

⁶ <https://spectrum.ieee.org/>

1.2 Why Python?

The developer intended to develop a Computer Programming Language with a focus on clean syntax. Towards this goal, Van Rossum, the founder of Python launched Computer Programming for Everybody (CP4E), at the time of stay at CNRI. His main intention was to make programming more accessible the people with the knowledge of literacy just like English literacy and Mathematics skills required.

Over the years language Python language has evolved and covers all multi-paradigm programming languages. As of now Python fully supports Object Oriented Programming (OOP) and Structured Programming and through extension, it supports design by contract and logic programming also.

The developer was influenced by Zen of Python⁷ which is a collection of 19 programming guidelines written by the software developer Tim Peters. He first wrote these 19 rules and mailed them to the Python mailing list in 1999. Since then, the original developer Guido van Rossum fulfilled the following principles, to make this language easy to understand. Such as: (a) Beautiful is better than ugly; (b) Explicit is better than implicit; (c) Simple is better than complex; (d) Complex is better than complicated; (e) Readability counts.

Python's versatility is difficult to match, and it's so flexible that it encourages experimentation. This keeps programmers from being pigeonholed into only building one type of application. You can learn just one language and use it to make new and different things.

Computer Weekly calls Python the most versatile programming language, noting that “Although there might be a better solution for any given problem, Python will always get the job done well” [1]. Other advantages of Python include:

- **Platform independent:** Like Java, you can use Python on various platforms, including macOS, Windows, and Linux. You'll just need an interpreter designed for that platform.
- **Fast development:** Because Python is dynamically typed, it's fast and friendly for development. Additionally, it uses asynchronous code to tackle situations and challenges faster because each unit of code runs separately.
- **Offers extensive libraries:** Its large library supports common tasks and commands. It also contains code that can be used for many different purposes, ranging from generating documentation to unit testing to CGI.
- **More flexible:** Python supports a variety of programming styles and has multiple paradigms. Not only is this optimal for programmers who enjoy flexibility, but it also makes it ideal for start-ups that might need to shift approaches abruptly.
- **Free and open-source:** You can download Python without any cost, and because it's so easy to learn and boasts one of the largest and most active communities—you should be able to start writing code in mere minutes.
- **Boost productivity:** NetGuru says that Python is more productive than Java because of how concise it is and because it's dynamically typed. Additionally, it has control capabilities and integration features that can make applications more productive [2].

1.3 Downside of Python

While Python is arguably one of the easiest and fastest languages to learn, it's also decidedly slower to execute because it's a dynamically typed, interpreted language, executed line-by-line. Python does extra work while executing the code, making it less suitable for use in projects that depend on speed. However, if speed isn't a sensitive issue, Python's slower nature won't likely be a problem. Other potential disadvantages include:

⁷ https://en.wikipedia.org/wiki/Zen_of_Python

- **It only executes one thread at a time:** Python has a Global Interpreter Lock that only lets one thread execute at a time, so if you're working on a multi-threaded CPU-bound program, it'll likely be even slower. Using multiprocessing programs instead of multithreaded programs can be an effective workaround.
- **It isn't mobile native:** Python can be effectively and easily used for mobile purposes, but you'll need to put a bit more effort into finding libraries that give you the necessary framework. Some examples include Kivy, which lets you use the same API to create mobile apps and software that you can run on Raspberry PI, Linux, and Windows.
- **It uses a large amount of memory:** If you're working on a project where many objects are active in RAM, this could present an issue for you. Switching to NumPy could be an effective workaround to reduce the amount of memory Python uses for each object.

1.4 What Python can do?

- **Automation:** it describes a wide range of technologies that reduce human effort by predetermining decision criteria, sub-process relationships, and related actions. In automation, Python is used for email automation such as a system engineer who wants to inform all the project members about the system maintenance. There are many popular libraries written in Python for these purposes namely PyAutoGUI, Pywinauto.
- **Data Analytics:** is a process of understanding and getting information from data of previous sales or advertisements. It is used to find meaningful patterns in data used for interpretation and communication purposes. Understanding user experience and sales are two key factors in areas rich with recorded information. Many packages such as PandasGUI, D-Tale, and Mito are the most popular packages used by data analytics engineers.
- **Databases:** relational databases like MySQL, SQLite, and Oracle are used for efficient data storage and management for any organizational growth. Each database requires a specific connector library to be installed in your Python environment to interact with it. Popular examples include sqlite3 for SQLite, mysql-connector-python for MySQL, and psycopg2 for PostgreSQL.
- **GUI:** also known as Graphical User Interface programming, involves creating applications with a visual interface that users can interact with through elements like buttons, text boxes, menus, and windows. This makes the software more user-friendly and accessible compared to command-line interfaces. Most popular Python libraries that offer cross platform developments are Tkinter, wxPython, PyQt.
- **Image Processing:** it refers to the process of enhancing the quality of an image or extracting information from an image. Any image processing tool offers a collection of computer algorithms to work with digital images. It has a wide range of applications such as Medical imaging, remote sensing, computer vision, security and surveillance. Python packages like scikit-image, OpenCV, and Pillow offer a great bunch of image processing algorithms to work with digital images.
- **Machine Learning and Text Processing:** there are the two sub-domains of Artificial Intelligence (AI) and nowadays any application with AI requires machine learning algorithms to make any decision. Text processing refers to the automated manipulation and analysis of electronic text. It involves transforming raw text data into a more usable format for various purposes, like: extracting information, cleaning and preparing data, summarizing text, classifying text and many more. Packages like NLTK, spaCy, Gensim, offer many text processing algorithms and scikit-learn, PyTorch, TensorFlow are preferable for machine learning and deep learning algorithms.
- **Web Scrapping:** also known as web harvesting or extracting data from web. Its application includes price comparison, market research, data analysis and content aggregation. The most popular Python libraries that offer web scrapping tools are Requests, BeautifulSoup, Selenium and Scrapy.

1.5 Evaluation of Python

The language name Python was named after the BBC TV show Monty Python's Flying Circus. In the late 1980s, the idea of Python Development was conceived by Guido van Rossum at Centrum Wiskunde & Informatica (CWI) in the Netherlands as a successor to the ABC programming language. Its implementation began in December 1989. Since then, Guido van Rossum carried the sole responsibility as Python dictator as a lead developer until 12 July 2018. In 2019 Python core developers elected a five-member group to lead future Python development.

The first version of Python was released in 1991 and codes were published at alt. sources. At this stage of development language already includes many important features such as classes with inheritance, exception handling, function, and the core datatypes list, dict, str, and so on. Initially, all the codes released in the module (about the module we have written a chapter) were inspired by Modula-3 language. Finally, in 1994 the primary discussion forum comp.lang.python was formed marking a milestone in the growth of Python's userbase.

To form the BeOpen Python Labs team all the core development team members moved to BeOpen.com in 2000. The initial development is carried out in CNRI and requested to release version 1.6 which has all the Python development at that point. Consequently, the release schedules for 1.6 and 2.0 had a significant amount of overlap. After the release of Python 2.0, all the Python Labs developers joined Digital Creations. Over the years Python goes through various releases and Table 1.2 highlights the details of all the previous and recent releases.

Table 1.2 Details of Python Releases with various versions and dates

Version	Date of Release	End of full support	End of security fixes
0.9	20-02-1991	29-07-1993	
1.0	26-01-1994	15-02-1994	
1.1	11-10-1994	10-11-1994	
1.2	13-04-1995	Unsupported	
1.3	13-10-1995	Unsupported	
1.4	25-10-1996	Unsupported	
1.5	03-01-1998	13-04-1999	
1.6	05-09-2000	September 2000	
2.0	16-10-2000	22-06-2001	
2.1	15-04-2001	09-04-2002	
2.2	21-12-2001	30-05-2003	
2.3	29-06-2003	11-03-2008	
2.4	30-11-2004	19-12-2008	
2.5	19-09-2006	26-05-2011	
2.6	01-10-2008	24-08-2010	29-10-2013
2.7	03-07-2010	01-01-2020	
3.0	03-12-2008	27-06-2009	
3.1	27-06-2009	12-06-2011	06-04-2012
3.2	20-02-2011	13-05-2013	20-02-2016

3.3	29-09-2012	08-03-2014	29-09-2017
3.4	16-03-2014	09-08-2017	18-03-2019
3.5	13-09-2015	08-08-2017	30-09-2020
3.6	23-12-2016	24-12-2018	23-12-2021
3.7	27-06-2018	27-06-2020	27-06-2023
3.8	14-10-2019	03-05-2021	Oct-2024
3.9	05-10-2020	17-05-2022	Oct-2025
3.10	04-10-2021	June-2024	Oct-2027
3.11	24-10-2022	June-2025	Oct-2028

Python 2.7 was the last release in 2.x series. In November 2014, it was announced that till 2020 the supports were provided and the developers are encouraged to move 3.0 as early as possible. Python 2.7 support ended on January 1, 2020, along with code freeze of 2.7 development branch. A final release, 2.7.18, occurred on April 20, 2020, and included fixes for critical bugs and release blockers. This marked the end of the life of Python 2.

Python 3.0 also known as Python 3000 or Py3K comes with the most necessary changes required in the language. At the time of release, it was not easy to retain the full backward compatibility with 2.x series. So, it broke the compatibility with Python 2 and much Python 2 code does not run unmodified on Python 3. Python supports dynamic typing and combines with the plan to change the semantics of certain methods, making it difficult for mechanical translation from 2.x to 3.0. Some of the translations are done automatically using a tool called 2to3.

Before the roll-out of Python 3.0, projects running in 2.x need to provide two different releases of all such projects. One version is completely written in 2.x for those systems running in Python 2.x and for the systems running in 3.x need to produce a release using 2to3.

1.6 Popular Python Libraries

Python's large standard library provides tools suited for many tasks and is commonly cited as one of its greatest strengths. It includes modules for creating graphical user interfaces, connecting to relational databases, generating pseudorandom numbers, arithmetic with arbitrary-precision decimals, manipulating regular expressions, and unit testing.

As of 14 November 2022, the Python Package Index (PyPI), the official repository for third-party Python software, contains over 415,000 packages with a wide range of functionality. Out of these here we have just listed top Python modules in Table 1.3 used for desktop applications, database operations, game development, web scraping, image processing, data science, machine learning, and natural language processing.

Table 1.3 List of Top ranked Python modules used for various purposes

Module	Used for	Web Address
PyQt5	Most popular Python framework for GUI.	https://riverbankcomputing.com/software/pyqt/intro
Tkinter	Another GUI toolkit comes as a pre-installed	https://wiki.python.org/moin/TkInter
wxPython	Cross-platform GUI Toolkit	https://wxpython.org/
PyGObject	Provides binding for GObject-based libraries such as GTK, GStreamer,	https://pygobject.readthedocs.io/en/latest/

	WebKitGTK, GLib, GIO, and many more	
Pmw	Used for building high-level compound widgets in Python using the Tkinter module.	https://pmw.sourceforge.net/doc/index.html
Tix	Tk Interface eXtension is a powerful set of user interface components that expands the capabilities of your Tcl/Tk and Python applications.	https://tix.sourceforge.net/
Libavg	It is widely considered one of the best libraries for developing user interfaces for modern touch-based devices, and its hardware acceleration is achieved through OpenGL and GPU shaders.	https://www.libavg.de/site/
PyForms	Python implementation of Windows Forms, which enables developers to create highly interactive interfaces for Windows GUI mode, Web mode, and Terminal mode.	https://pyforms.readthedocs.io/en/v4/
Pygame	set of Python modules designed for writing video games.	https://www.pygame.org/wiki/about
PYGLET	cross-platform windowing and multimedia library for Python	https://pyglet.org/
PyOpenGL	is the most common cross-platform Python binding to OpenGL and related APIs.	https://pyopengl.sourceforge.net/
MySQLdb	Used to working with MySQL database	https://sourceforge.net/projects/mysql-python/
PyGreSQL	Interface to connect with PostgreSQL database.	http://www.pygresql.org/about.html
Gadfly	A relational database implemented in Python	https://gadfly.sourceforge.net/
SQLAlchemy	SQL toolkit and Object Relational Mapper that gives application developers full power.	https://www.sqlalchemy.org/
KInterbasDB	Supports open-source relational database Firebird	https://kinterbasdb.sourceforge.net/
BeautifulSoup	Used for web scraping and converts anything to Unicode	https://www.crummy.com/software/BeautifulSoup/
Scrape	Used for web scraping	http://zesty.ca/python/scrape.html
mechanize	Stateful programmatic web browsing in	https://wwwsearch.sourceforge.net

	Python	/mechanize/
Scrapy	Used to extracting the data that you need from websites	https://scrapy.org/
Pyquery	allows you to make jquery queries on xml documents	https://pythonhosted.org/pyquery/
Selenium	is a suite of tools used to automate a browser	https://www.selenium.dev/
Requests	Requests allows you to send HTTP/1.1 requests extremely easily.	https://pypi.org/project/requests/
Googlemaps	Provides google map API services	https://pypi.org/project/googlemaps/
OpenCV	Used for computer vision and machine learning with artificial intelligence	https://opencv.org/
Scikit-Image	Image processing library	https://scikit-image.org/
Scipy	Used for mathematical and scientific computation	https://scipy.org/
NumPy	Library for numerical analysis	https://numpy.org/
matplotlib	Specialized for 2-d plots as a multi-platform data visualization on NumPy arrays	https://matplotlib.org/
TensorFlow	Used for deep learning and data science projects, developed by the Google Brain team	https://www.tensorflow.org/
Pandas	Widely used Python libraries for Data Science	https://pandas.pydata.org/
Scikit-learn	The machine-learning library provides a variety of useful machine-learning algorithms, and it is designed to be interpolated into SciPy and NumPy.	https://scikit-learn.org/stable/
Keras	Used for neural network modules, similar to TensorFlow	https://keras.io/
PyTorch	Created by Facebook AI for scientific computing package relies on the power of GPU.	https://pytorch.org/
Theano	A numerical computation Python library specifically developed for machine learning and deep learning.	https://github.com/Theano/Theano no
Microsoft CNTK	The open-source deep-learning library is used to implement distributed deep learning and machine learning tasks.	https://learn.microsoft.com/en-us/cognitive-toolkit/
NLTK	Best python library for Natural Language Processing	https://www.nltk.org/

SpaCy	SpaCy enables developers to create applications that can process and understand huge volumes of text	https://spacy.io/
Genism	The fastest library for training of vector embeddings.	https://radimrehurek.com/gensim/
Bertnlp	performs various NLP tasks using Bidirectional Encoder Representations from Transformers (BERT) related models.	https://pypi.org/project/bertnlp/
TextBlob	Used for processing textual data. It provides a simple API for diving into common NLP tasks such as part-of-speech tagging, noun phrase extraction, sentiment analysis, classification, translation, and more.	https://textblob.readthedocs.io/en/dev/
PyNLPI	PyNLPI can be used for basic tasks such as the extraction of n-grams and frequency lists, and to build a simple language model.	https://pynlpl.readthedocs.io/en/latest/
Pattern	Web mining module for python	https://github.com/clips/pattern
Polyglot	natural language pipeline that supports massive multilingual applications	https://polyglot.readthedocs.io/en/latest/
vaderSentiment	is a lexicon and rule-based sentiment analysis tool that is specifically attuned to sentiments expressed in social media, and works well on texts from other domains.	https://pypi.org/project/vaderSentiment/
pySonic	is a Python wrapper around the high performance, cross platform FMOD sound library.	https://pysonic.sourceforge.net/
pyMedia	is a Python module for wav, mp3, ogg, avi, divx, dvd, cdda etc files manipulations.	http://pymedia.org/

1.7 Python Implementations

CPython was the first and referenced implementation written using C89 standards. Later Python 3.11 uses C11 with several C99 features. CPython compiles every Python program into intermediate bytecode and executed by its virtual machine. Bytecode also called portable code is a form of instruction set used for efficient execution by interpreter. CPython is distributed with large set of libraries mostly written in C and native Python and publicly available for many platforms such as Windows and most modern UNIX like systems. Python have many other implementations, includes:

- PyPy is a fast, compliant interpreter of Python 2.7 and 3.8 comes with just-in-time compiler for speed improvement over CPython [3].

- Stackless Python is a significant fork of CPython that implements microthreads; it does not use the call stack in the same way, thus allowing massively concurrent programs [4].
- For microcontroller including Lego Mindstorms EV3, Micro Python and Circuit Python are the two Python 3 variants [5].
- Pyston is a variant of the Python runtime that uses just-in-time compilation to speed up the execution of Python programs [6].
- Cinder is a performance-oriented fork of CPython 3.8 that contains a number of optimizations including bytecode inline caching, eager evaluation of coroutines, a method-at-a-time JIT, and an experimental bytecode compiler.

In addition to the above list, it also has some other implementations designed as cross compiler to other languages. These includes:

- Brython⁸, Transcrypt⁹ and Pyjs compile Python to JavaScript.
- Cython compiles (a superset of) Python 2.7 to C and the resulting code is also usable with Python 3.
- Nuitka¹⁰ compiles Python into C.
- Numba¹¹ uses LLVM to compile a subset of Python to machine code.
- Pythran¹² compiles a subset of Python 3 to C++11 only.
- RPython¹³ can be compiled to C and is used to build the PyPy interpreter of Python.
- MyHDL¹⁴ is a Python-based hardware description language (HDL), that converts MyHDL code to Verilog or VHDL code.

These are some older implementations that didn't support any of the Python 3.x syntax. Such as:

- Google's Grumpy¹⁵ with latest release in 2017, translates Python 2 to Go.
- IronPython¹⁶ allows running Python 2.7 and 3.4 programs on the .NET Common Language Runtime.
- Jython¹⁷ compiles Python 2.7 to Java bytecode, allowing the use of the Java libraries from a Python program.
- Pyrex¹⁸ and Shed Skin compile to C and C++ respectively.

*****END OF CHAPTER*****

⁸ <https://brython.info/>

⁹ <https://www.transcrypt.org/>

¹⁰ <https://nuitka.net/>

¹¹ <https://numba.pydata.org/>

¹² <https://pythran.readthedocs.io/en/latest/>

¹³ <https://pypi.org/project/rpython/>

¹⁴ <https://myhdl.org/>

¹⁵ <https://github.com/google/grumpy>

¹⁶ <https://ironpython.net/>

¹⁷ <https://www.jython.org/>

¹⁸ <https://www.cs.cantbury.ac.nz/greg.ewing/python/Pyrex/>